

Scenario: Library Book Management System

Background:

Sarah has been hired to build a small application for a local library. The library wants to manage its book records. Each record includes the book's title, author, and number of copies available. The application should allow adding new books, grouping books by author, analyzing availability using LINQ, and filtering books based on minimum copies.

Functionalities

In class Program:

```
public static SortedDictionary<int, BookRecord> bookRecords;
```

This sorted dictionary is already provided. It stores book records with an auto-incremented key.

In class BookRecord, implement the following properties:

Data Type	Property Name
string	Title
string	Author
int	Copies

In class LibraryUtility, implement the following methods:

Method 1:

```
public void AddBookRecord(string title, string author, int copies)
```

Adds a new book record to the bookRecords dictionary.
The key is set to one more than the current number of items in the dictionary.

Method 2:

Groups all book records by their author.

Returns a dictionary where:

Key = Author name

Value = List of book records under that author

Method 3:

```
public Dictionary<string, double> GetAverageCopiesPerAuthor()
```

Uses LINQ to calculate the average number of copies for each author.

Method 4:

```
public BookRecord GetBookWithMaxCopies()
```

Uses LINQ to find the book with the highest number of copies available.

Method 5:

```
public List<BookRecord> FilterBooksByCopies(int minCopies)
```

Returns a list of books whose copies are greater than or equal to minCopies.

Sample Input/Output

1. Add Book Record
2. Group Records By Author
3. Show Average Copies Per Author
4. Show Book With Maximum Copies
5. Filter Books By Minimum Copies
6. Exit

Sample Run

Enter your choice

1

Enter book title

C# Basics

Enter author

John Doe

Enter copies

5

Book record added successfully

Enter your choice

1

Enter book title

Advanced LINQ

Enter author

Jane Smith

Enter copies

8

Book record added successfully

Enter your choice

1

Enter book title

ASP.NET Core

Enter author

John Doe

Enter copies

3

Book record added successfully

Enter your choice

2

John Doe

C# Basics - 5

ASP.NET Core - 3

Jane Smith

Advanced LINQ - 8

Enter your choice

3

Average Copies:

John Doe - 4.0

Jane Smith - 8.0

Enter your choice

4

Book With Maximum Copies:

Advanced LINQ - 8

Enter your choice

5

Enter minimum copies

4

Books with copies ≥ 4 :

C# Basics - 5

Advanced LINQ - 8

Enter your choice

6

Thank you

in c# don't use linq

E-Commerce: Order Intake Manager

Arjun manages a busy online store where new customer orders arrive constantly. He prefers to review the most recent orders first. Utilise your C# programming skills to help Arjun track each order as it is received and process them in reverse order of arrival using a Stack. Provide functionality to add new orders, get the most recent order, and remove it when processed.

Functionalities:

In class Program, implement the below given public static property.

Data Type

Property Name

Stack<Order>

OrderStack

In class Order, implement the below given properties and methods.

Data Type

Property Name

int

OrderId

string

CustomerName

string

Item

Method

Description

```
public Stack<Order> AddOrderDetails(int orderId, string customerName, string item)
```

Add the order details to OrderStack and return it.

```
public string GetOrderDetails()
```

Get the most recently added order details from OrderStack and return them as a string separated by one space.

Format: OrderId" "CustomerName" "Item

```
public Stack<Order> RemoveOrderDetails()
```

Remove the most recently added order from OrderStack and return the remaining stack.

In Program class - Main method,

Get the values from the user.

Call the methods accordingly and display the result as per the Sample Output.

Note:

Keep the methods and classes as public.

Please read the method rules clearly.

Do not use Environment.Exit() to terminate the program.

Do not change the given code template.

Summary:

This task reinforces working with the generic collection Stack<T> in C#, which represents a last-in, first-out (LIFO) collection.

You are processing an inventory list where each element represents the quantity of an item in a warehouse slot.

A value of 0 indicates a placeholder slot (empty or blocked) that should be moved to the end of the list for packing efficiency.

All non-zero quantities must retain their original relative order to preserve upstream sorting and picking logic.

Your task is to rearrange the array in-place so that:

All non-zero elements keep their relative order.

All zeros are moved to the end.

Do this with O(1) extra space and preferably O(n) time.

Think of it like compressing all real items to the front while sliding all empty slots to the back—without shuffling the items among themselves.

Input / Output Format

Input:

An integer array arr (length n, where $0 \leq n \leq 10^5$), elements can be any 32-bit signed integers.

In the base version, treat only 0 as the placeholder to push.

Output:

Modify the array in place so that all non-zero elements appear first (in their original order), followed by all zeros.

Constraints

Time complexity target: O(n)

Space complexity: O(1) additional space (ignore recursive stack if you choose recursion, though iterative is expected)

You must not create a new array to collect the elements (no Where(...).ToArray() reassignments).

Examples

Example 1

Input: [0, 1, 0, 3, 12]

Output: [1, 3, 12, 0, 0]

Explanation: Non-zeros (1, 3, 12) keep order; zeros slide to the end.

Example 2

Input: [4, 0, 5, 0, 0, 7, 8]

Output: [4, 5, 7, 8, 0, 0, 0]

Example 3

Input: [0, 0, 0]

Output: [0, 0, 0]

Example 4

Input: []

Output: []

Example 5 (negatives allowed)

Input: [0, -2, 0, -3, 4]

Output: [-2, -3, 4, 0, 0]

Edge Cases to Consider

All zeros or no zeros.

Single element arrays: [0], [5].

Already stabilized arrays: [1, 2, 3].

Large arrays near limits: performance and in-place writes.

Arrays with consecutive zeros in the middle and end.